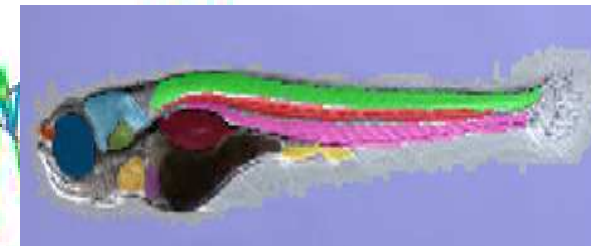


Object Detection Quality Evaluation

IoU

Intersection Over Union

**Occlusion
Map**



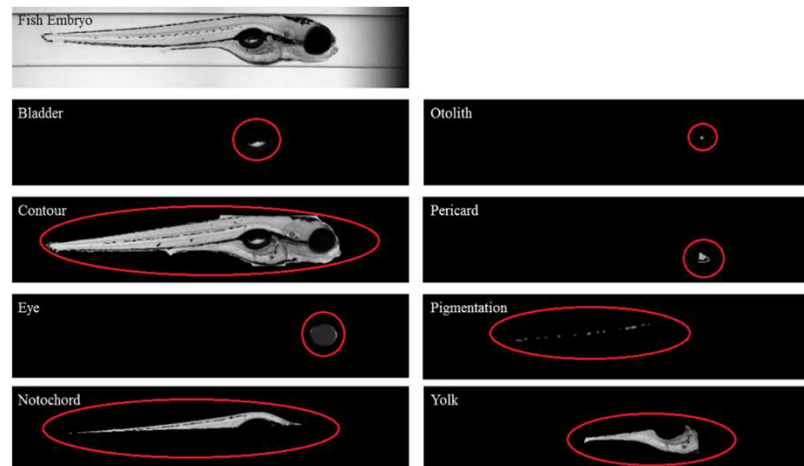
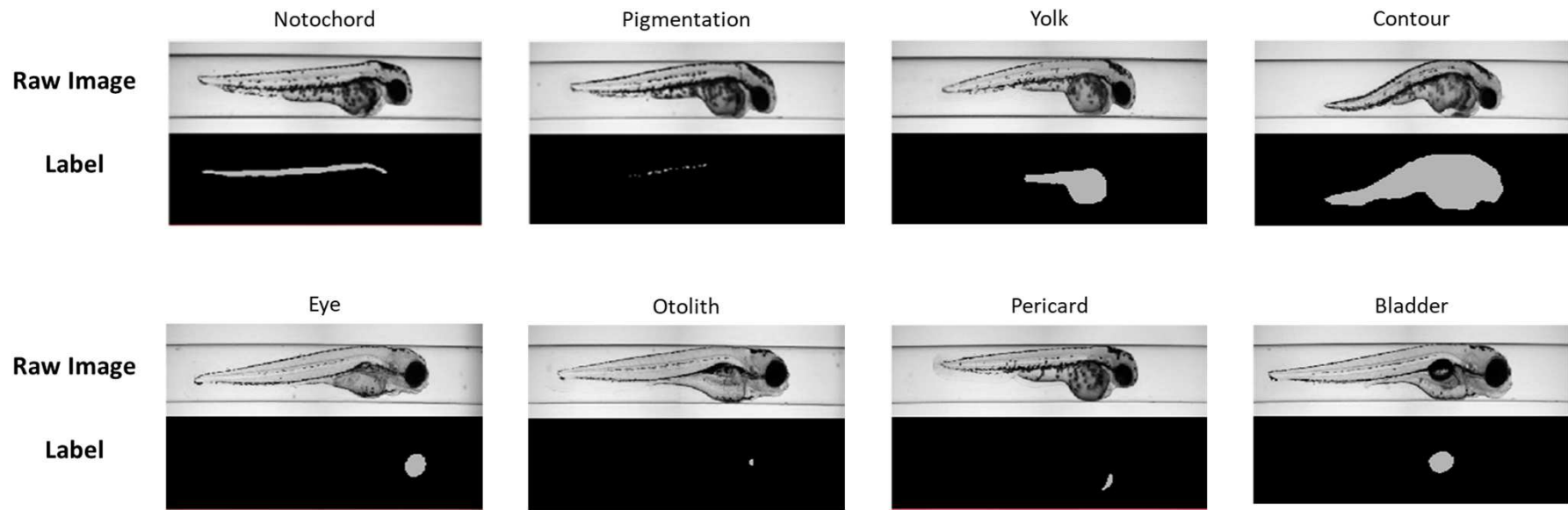
Software Engineering and Data Science
SEIS 766: Vision A.I.
Dr. Chih Lai

Outline

- Object Detection, Fast RCNN
- Object Detection, Semantic Segmentation, Labeling, Applications
- Encode + Decode CNN, Transposed Convolution, Pre-Trained CNN
 - Example applications
-  Quality Evaluation of Object Detection
- **Augmentation Pitfalls**
- Identifying Morphological Changes in Organs w/ Occlusion Map

Labels for Different Organs

- Imperfect labels from software tools so editing from biologists

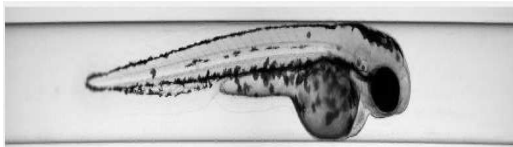


images+labels provided
by Dr. Stefan Scholz

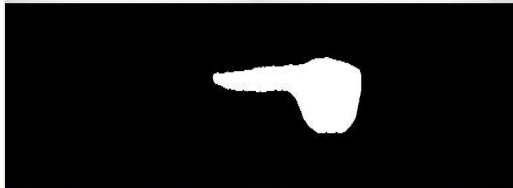
Identifying Organ **PIXELS** in Zebrafish **Embryo(??)** Images

Yolk

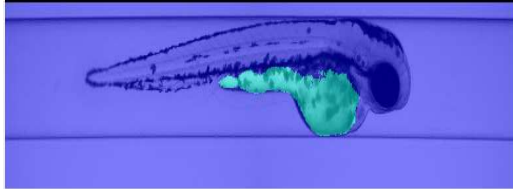
Raw



Labels



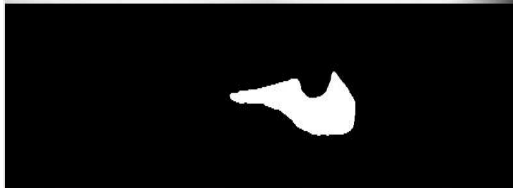
Detection



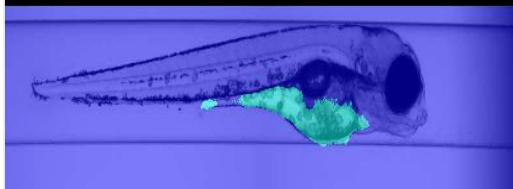
Raw



Labels



Detection



Notochord

\\VAI_Slide_FishEmbryo\

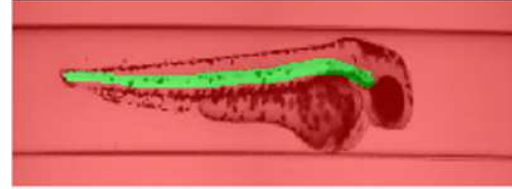
Raw



Labels



Detection



Raw



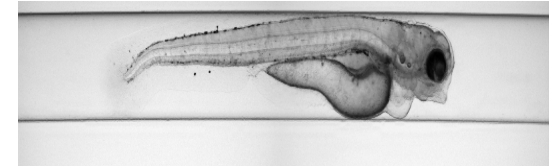
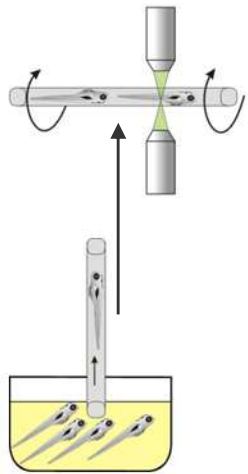
Labels



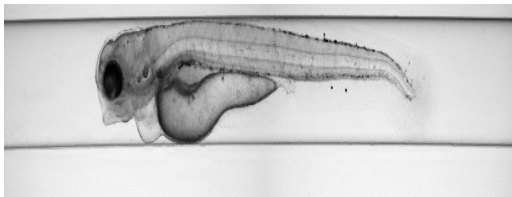
Detection



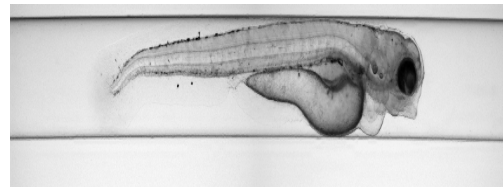
Taking Zebrafish Embryo Images from Microscope



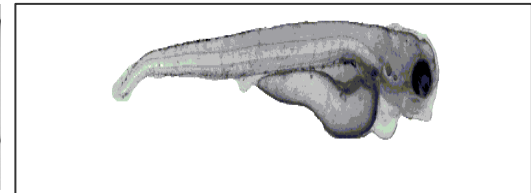
Original Image



Flipped Image



Preprocessed Image



First Try → One Model for Multiple Organs (Multiple Label Classes)

- Cool... but take **looo..ooong** to train 

Also, images are all in different orientations!!!

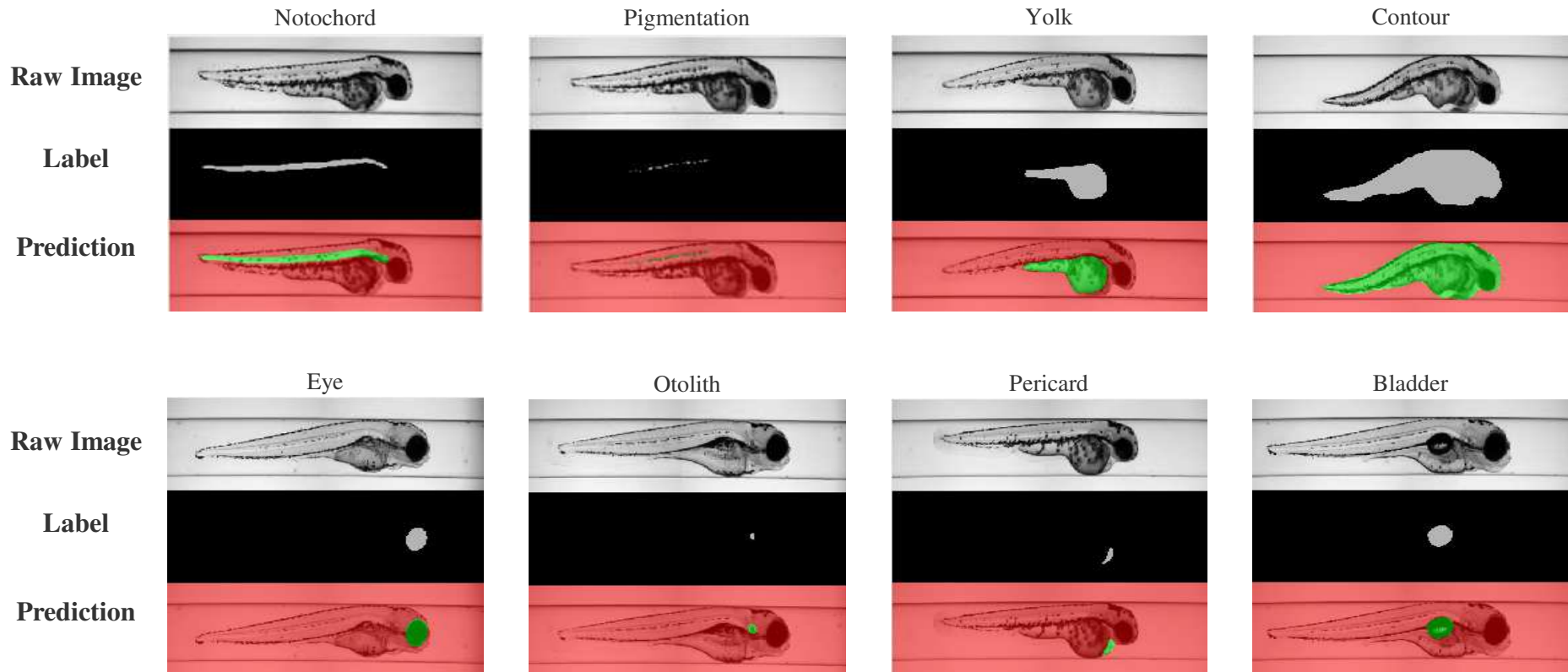


One Model for One Organ

- Easier to train w/ training time varies from hours to days
- Easier to evaluate & understand which organ is easy / hard

Images are all in
SAME orientations!!!

HOW???



- Looks Good, Feels Good... BUT, what is the **Quality** of Segmentation??
- **Use confusion Matrix? WHY NOT???**

Outline

- Object Detection, Fast RCNN
- Object Detection, Semantic Segmentation, Labeling, Applications
- Encode + Decode CNN, Transposed Convolution, Pre-Trained CNN
-  Quality Evaluation of Object Detection
- **Augmentation Pitfalls**
- Identifying Morphological Changes in Organs w/ Occlusion Map

Intersection Over Union (IoU)

- IoU for evaluating quality of object allocating

- $\text{IoU} = \frac{\text{Yellow}}{\text{Union}(\text{Blue}, \text{Red})}$
 $\text{intersect} / \text{Label, Detection}$

$$\text{IoU} = \frac{\text{Intersect}(\text{target}, \text{detected})}{\text{Union}(\text{target}, \text{detected})}$$

- Correct if $\text{IoU} \geq T$ (i.e. $T = 0.58$)
 - Maximum $\text{IoU} = 1$ IF **detection** area totally overlaps with **label** area



`metrics = evaluateSemanticSegmentation(detected, target);`

`loss = tf.keras.metrics.IoU(num_classes=3,
 target_class_ids=[1])
 loss.update_state(y_true, y_pred)`

GlobalAccuracy	MeanAccuracy	MeanIoU	WeightedIoU	MeanBFScore
0.90624	0.95085	0.61588	0.87529	0.40652
✗	✗			



IOU Keras Solution



- IOU for evaluating quality of object allocating

$$\text{IOU} = \frac{\text{Yellow}}{\text{Union}(\text{Blue}, \text{Red})} = \frac{\text{intersect}}{\text{Label, Detection}}$$

$$\text{IoU} = \frac{\text{Intersect}(\text{target}, \text{detected})}{\text{Union}(\text{target}, \text{detected})}$$

```
metrics = evaluateSemanticSegmentation(detected, target);
```

GlobalAccuracy	MeanAccuracy	MeanIoU	WeightedIoU	MeanBFScore
0.90624	0.95085	0.61588	0.87529	0.40652



```
intersect = np.logical_and(Y, y_hat)
union     = np.logical_or(Y, y_hat)
iou_2    = np.sum(intersect) / np.sum(union)
```

```
IoU_2classes = jaccard(BW1, BW2)
```

```
y_true = [
    [1, 0],
    [1, 0]
]
y_pred = [
    [0, 1],
    [1, 0]
]
```

```
keras_cv.bounding_box.compute_iou(
    boxes1, boxes2,
    bounding_box_format,
    use_masking=False, mask_val=-1,
    images=None, image_shape=None,
)
```

```
# target_class_ids indicates class/classes you want to calculate IoU on
loss = tf.keras.metrics.IoU(num_classes=-2, target_class_ids=[1])
loss.update_state(y_true, y_pred)
print(loss.result().numpy())
```

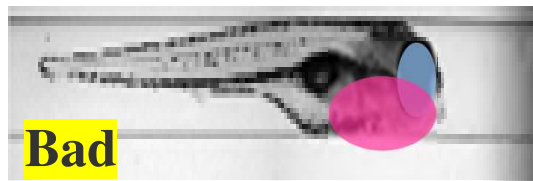


Jaccard Index = 0.72158

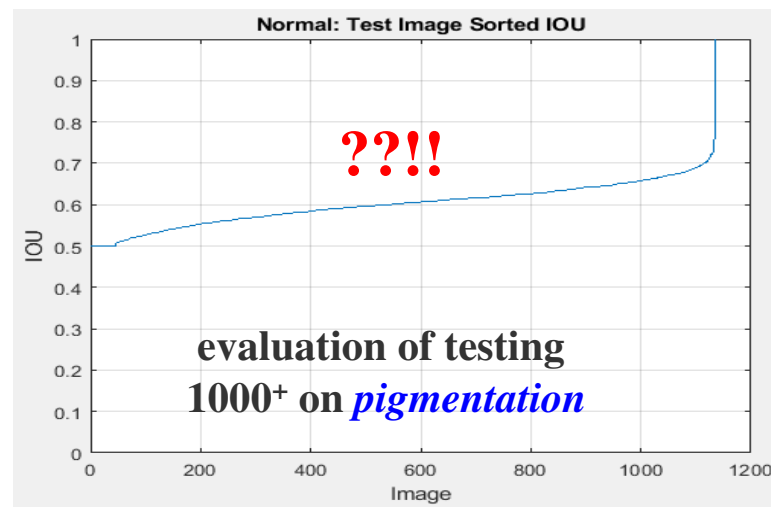
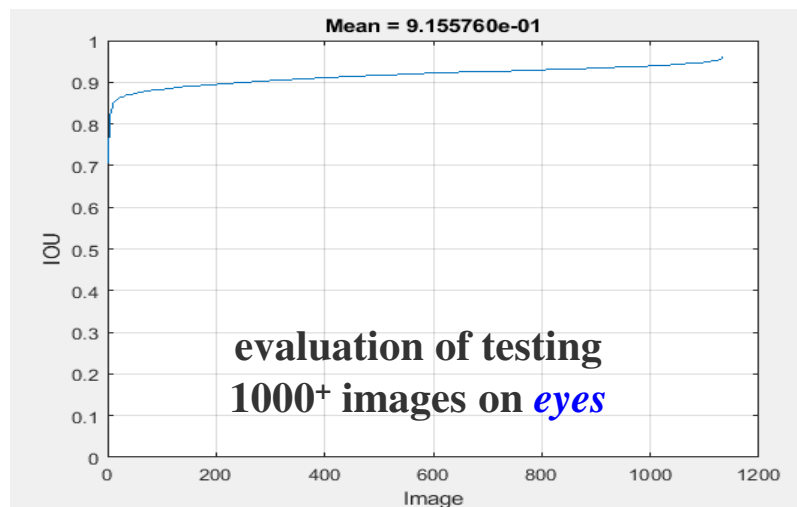
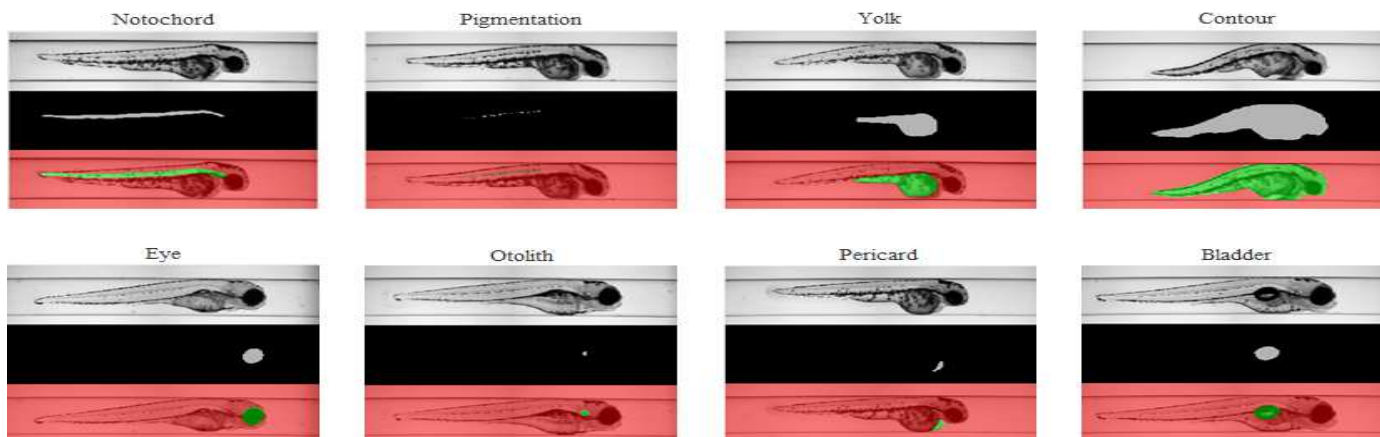


<https://www.mathworks.com/help/images/ref/jaccard.html>

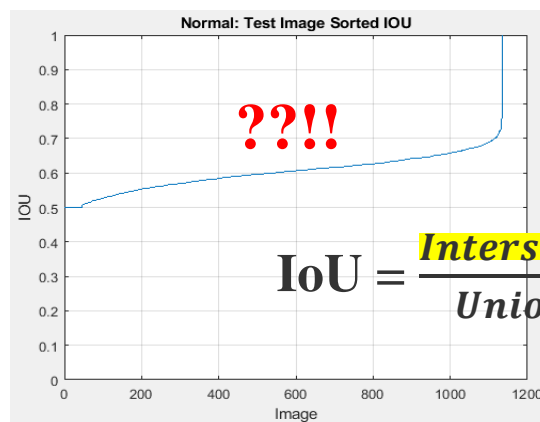
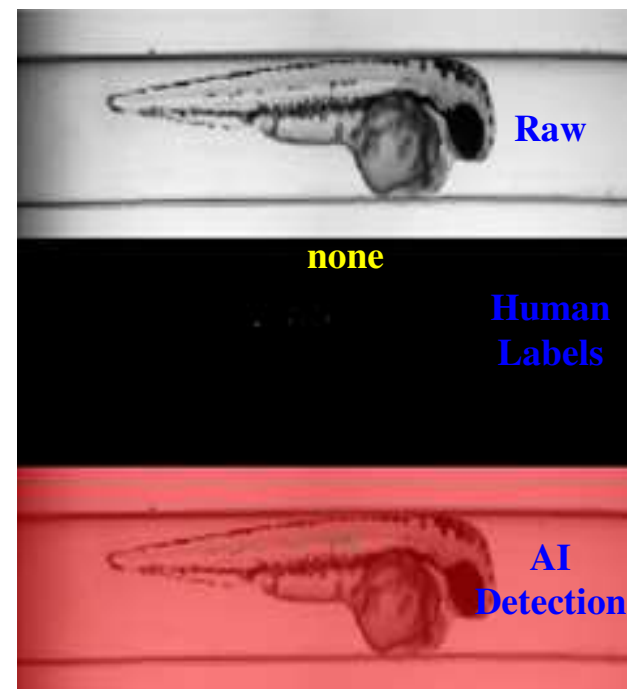
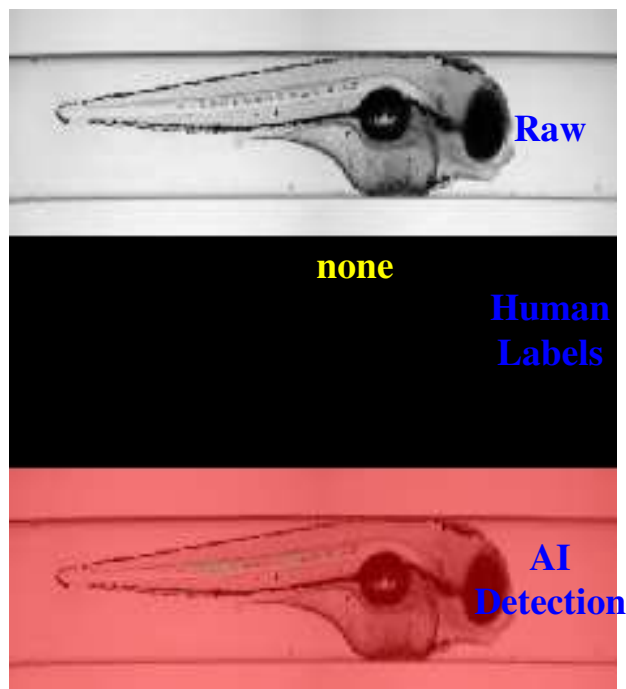
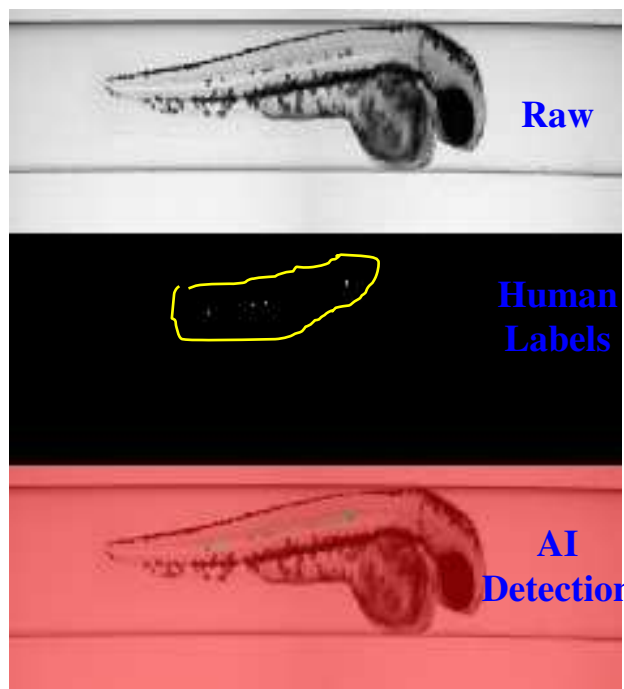
IoU Case Study, “target area” and “detected area”



$$\frac{\text{intersect}(\text{target}, \text{detected})}{\text{union}(\text{target}, \text{detected})}$$



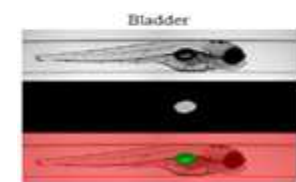
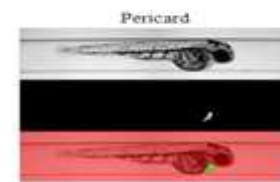
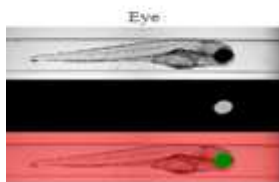
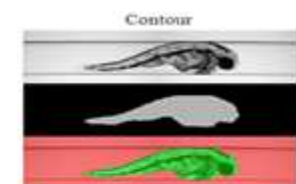
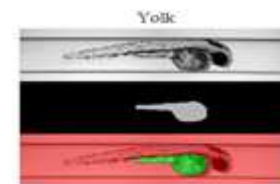
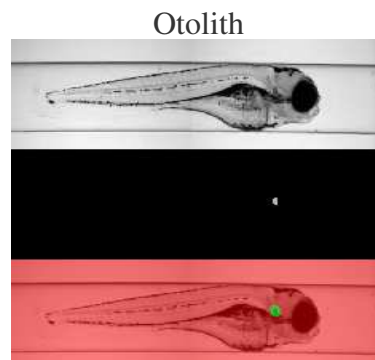
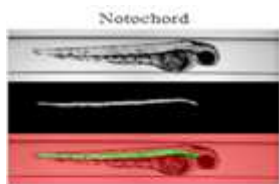
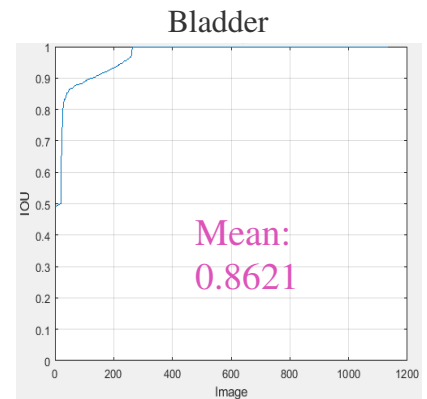
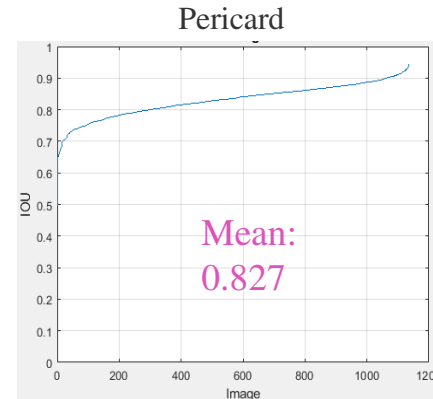
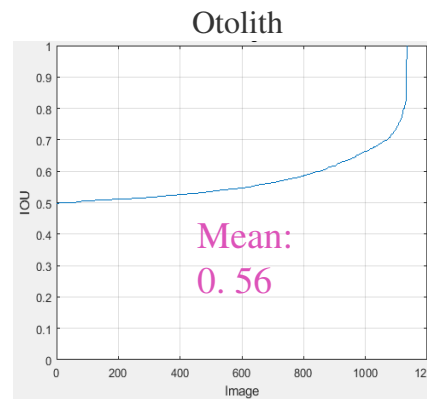
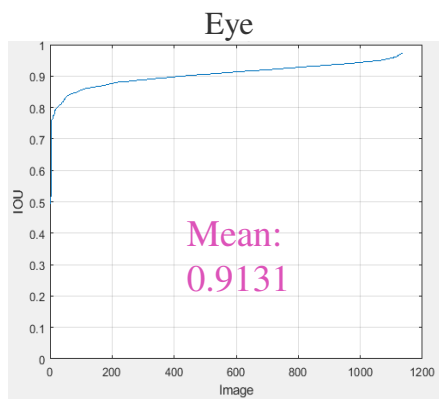
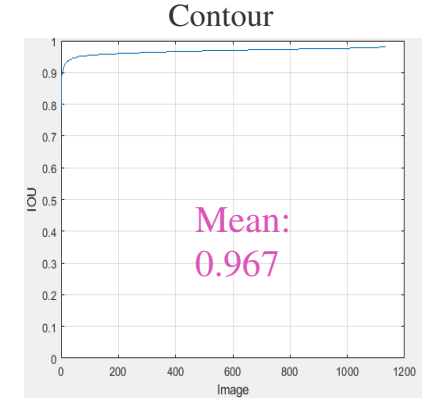
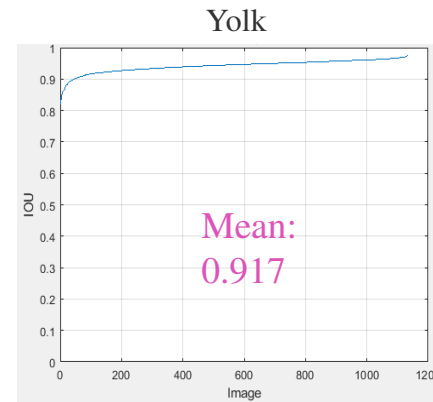
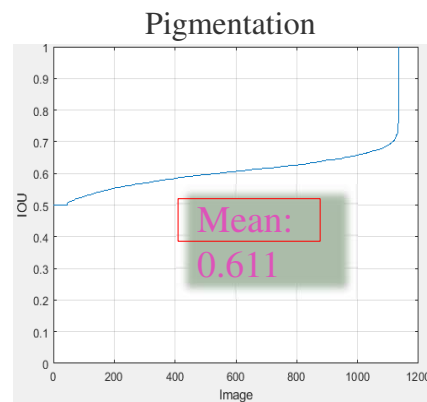
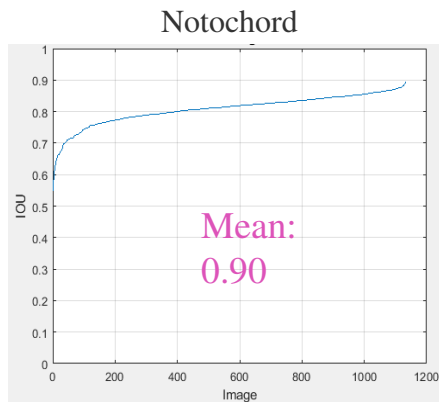
Is Deep Learning Better Than Human (Rules)? Pigmentation



$$\text{IoU} = \frac{\text{Intersect}(\text{target}, \text{detected})}{\text{Union}(\text{target}, \text{detected})}$$

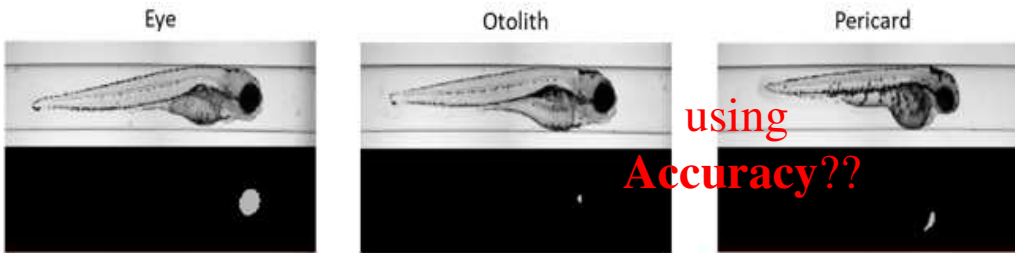
Overall Model Performance on All Organs

Created in 2018 by Dr. Lai's
Research Assistant Avani Saklecha

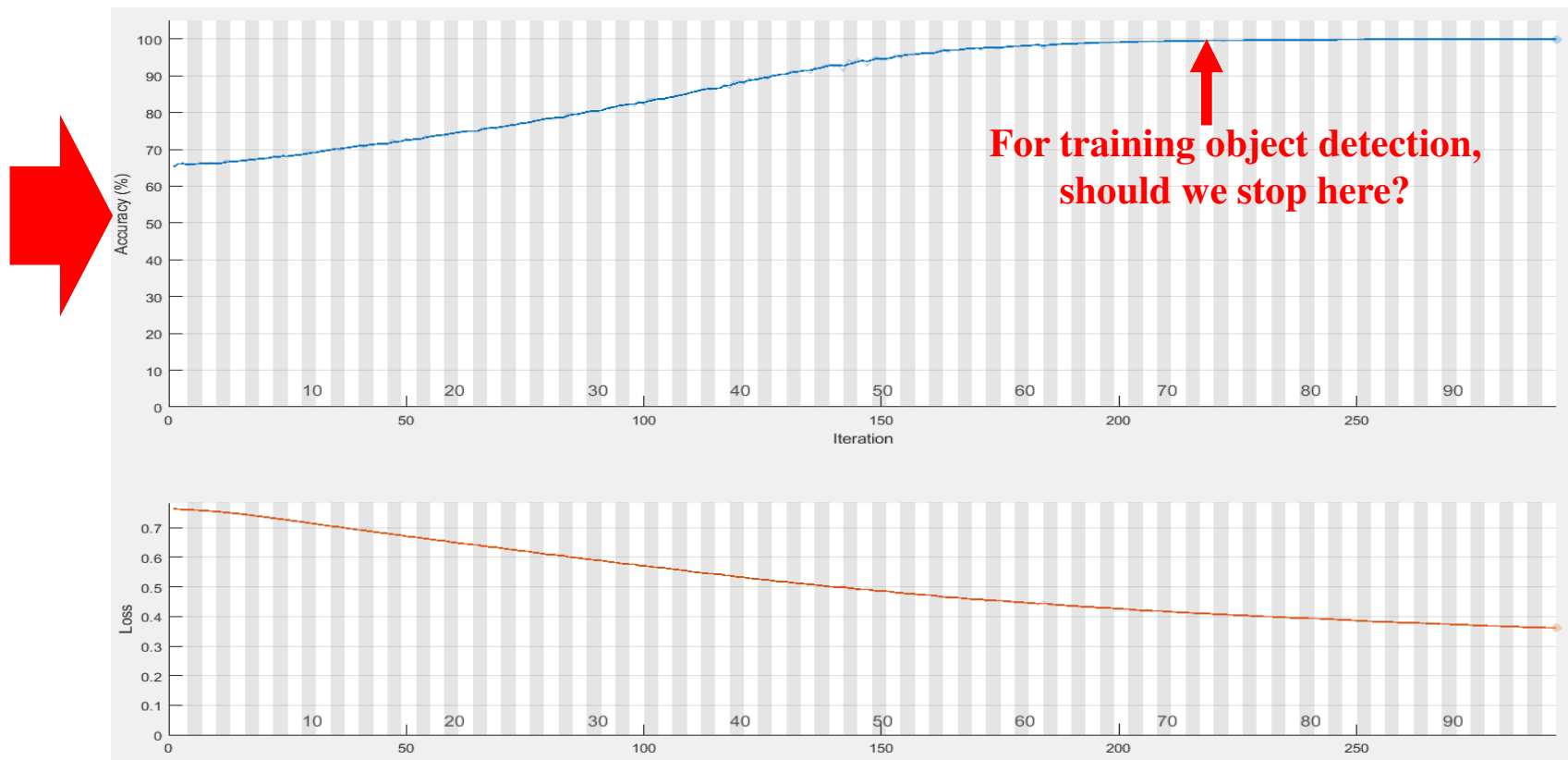


When To Stop Training Object Detection? Using Accuracy??

- When to stop? **check test data!!**



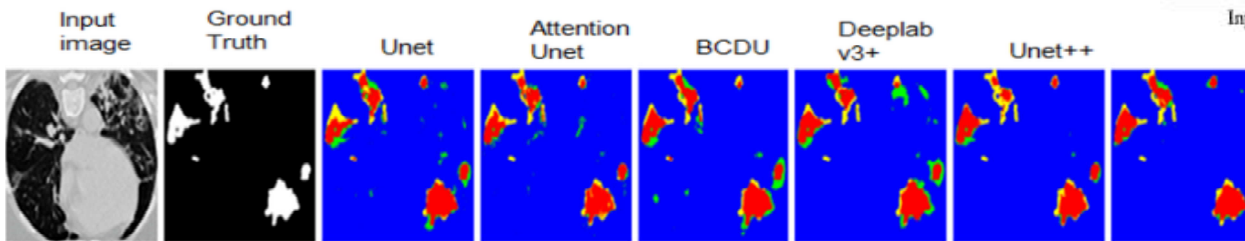
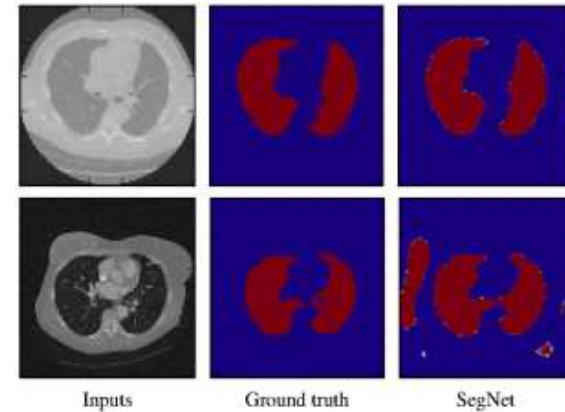
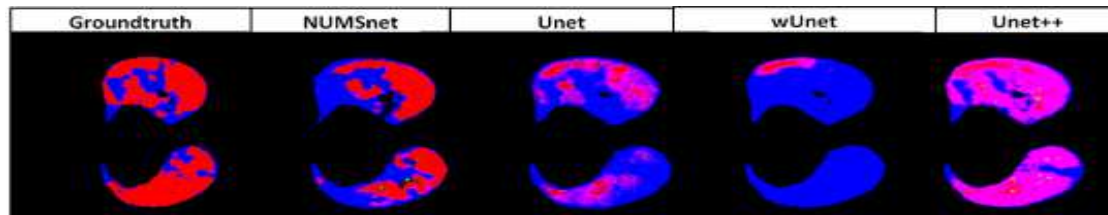
```
from tensorflow.keras.metrics import MeanIoU  
iou_metric = MeanIoU(num_classes=3)  
model.compile(  
    loss='categorical_crossentropy',  
    metrics=[iou_metric])
```



Noise Everywhere!!!

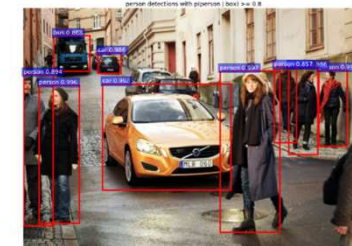
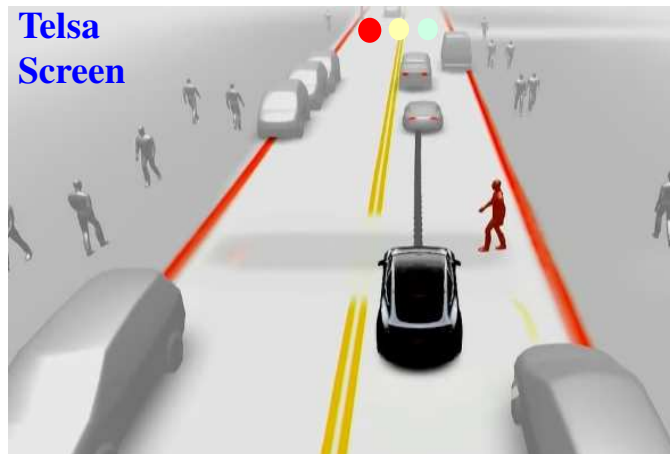
- Clean up w/ *morphological* operations (traditional image processing) Or...
- *Denoising Autoencoder*

2023: <https://www.mdpi.com/2078-2489/14/6/333>



2022: https://www.researchgate.net/publication/349517557_Contour-aware_semantic_segmentation_network_with_spatial_attention_mechanism_for_medical_image

<https://youtu.be/hxczTe9U8jY>

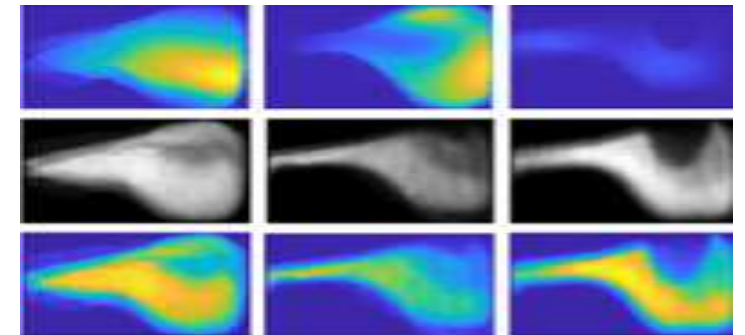
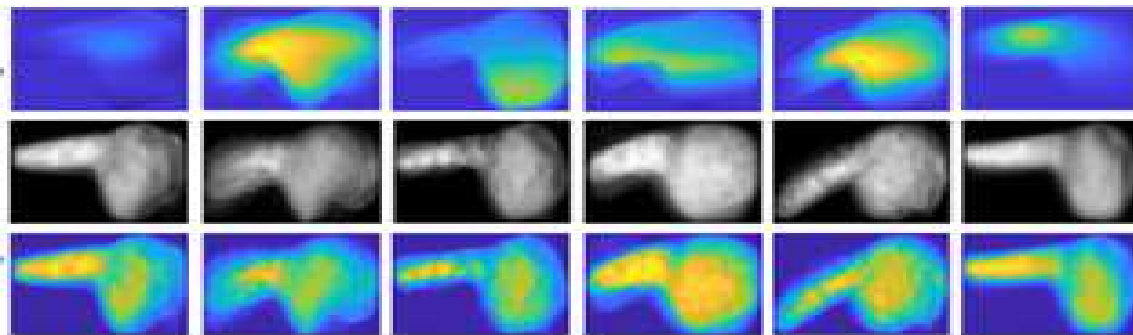
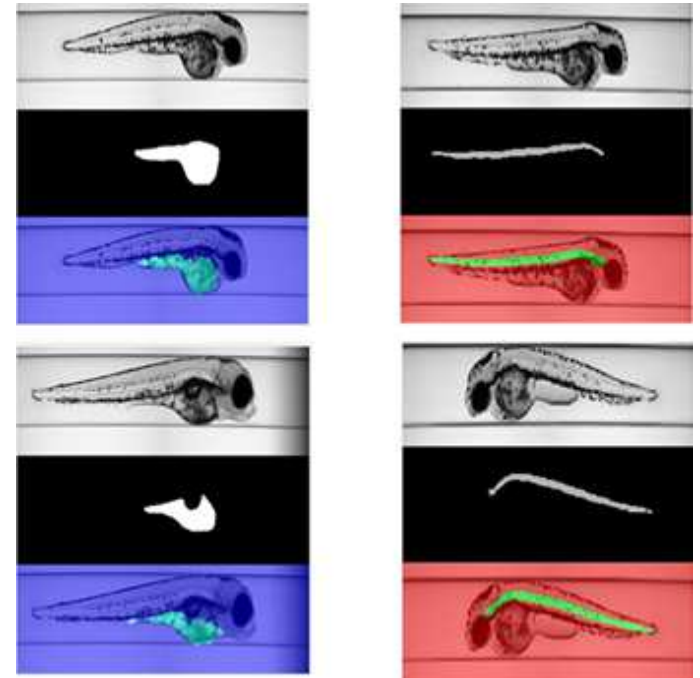


Outline

- Object Detection, Fast RCNN
- Object Detection, Semantic Segmentation, Labeling, Applications
- Encode + Decode CNN, Transposed Convolution, Pre-Trained CNN
- Quality Evaluation of Object Detection
- Augmentation Pitfalls
- Identifying Morphological Changes in Organs w/ Occlusion Map

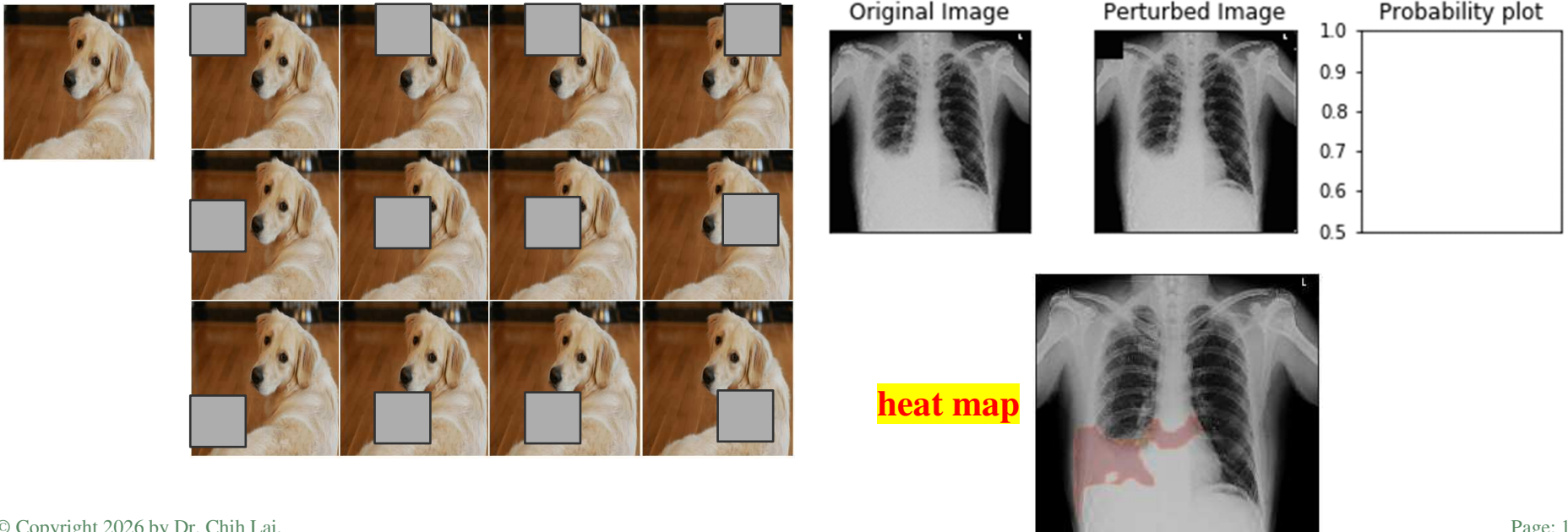


Occlusion Map

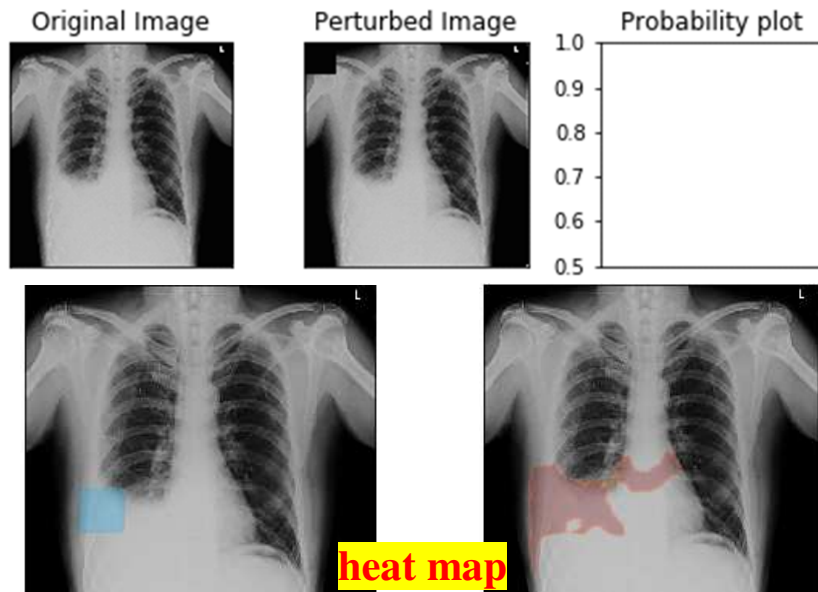


Occlusion Sensitive Map (**OS-Map**)

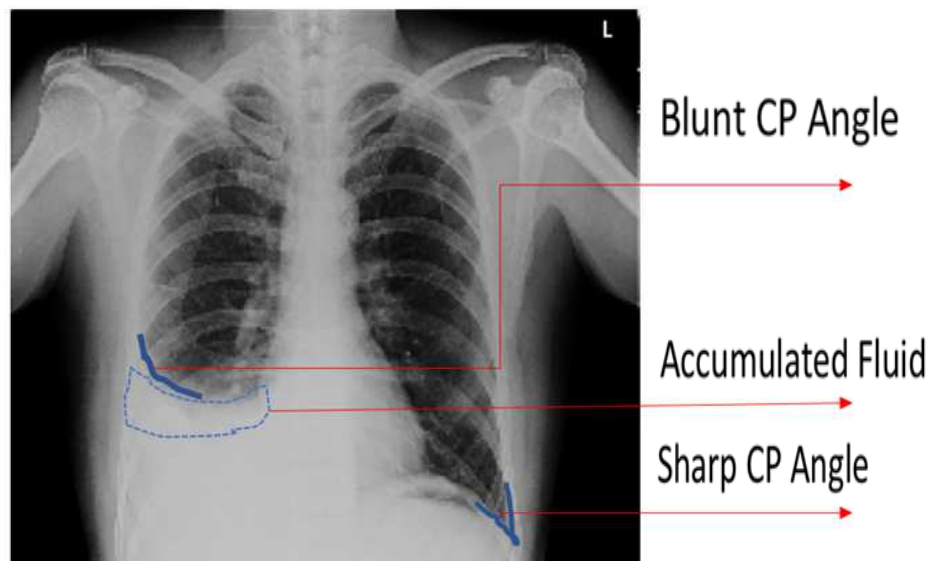
- Train an ordinary CNN classifier on your images for predicting ***Y***
- Use moving mask to occlude parts of img & check class probs. changes as a map
 - Move the occluding mask across each input image
 - Compute changes in classification probability at each mask location, and collect
 - Mark degrees of changes in original image as heat map to identify important parts for
- Resulting ***occlusion map*** identifies the most impact parts based on scores
 - Map areas with higher changes = image regions contribute more to ***classification***



Occlusion Sensitive Map (OS-Map)

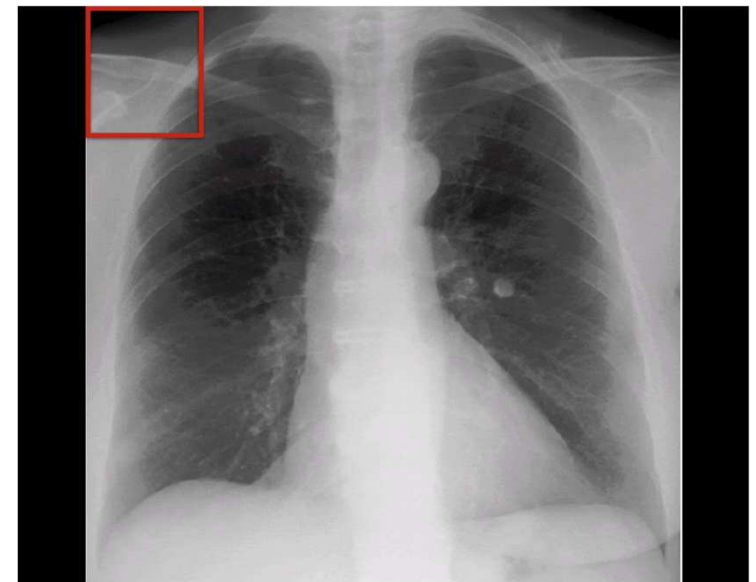


<https://blogs.mathworks.com/deep-learning/2017/12/15/network-visualization-based-on-occlusion-sensitivity/>



https://blog.gure.ai/notes/visualizing_deep_learning

© Copyright 2026 by Dr. Chih Lai,



<https://gifer.com/en/JV0w>

Easy OS-Map in Matlab

- Resulting **occlusion map** identifies the most impact parts based on predict probs.
 - Map areas with higher changes = image regions contribute more to **classification**
 - Simply call “**occlusionSensitivity(...)**”

<https://www.kaggle.com/code/blargl/simple-occlusion-and-saliency-maps>
<https://github.com/topics/occlusion-sensitivity>

```
net = googlenet;
```

```
X = imread("sherlock.jpg");
```

```
inputSize = net.Layers(1).InputSize(1:2);
```

```
X = imresize(X,inputSize);
```

```
imshow(X)
```

```
% % First, train a CNN classifier, “net”
```

```
label = classify(net, X) % golden retriever
```

```
scoreMap = occlusionSensitivity(net, X, label);
```

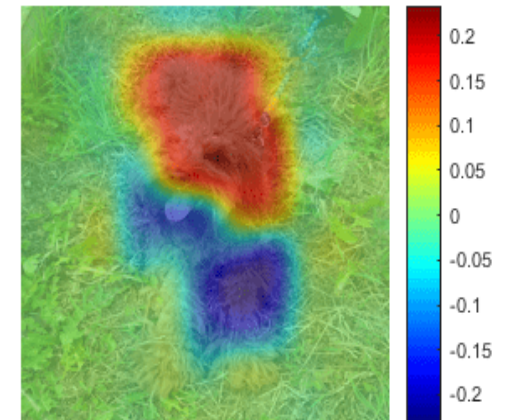
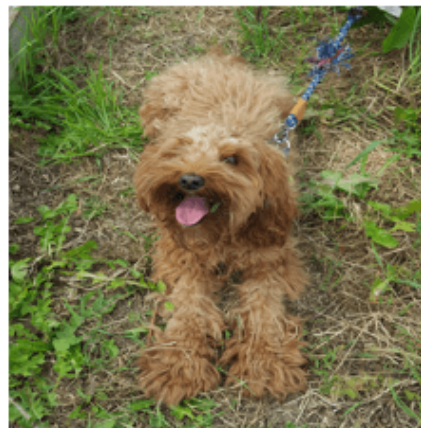
```
figure, imshow(X), hold on
```

```
imagesc(scoreMap, 'AlphaData', 0.5);
```

```
colormap jet
```



<https://www.mathworks.com/help/deeplearning/ref/occlusionsensitivity.html>



<https://www.mathworks.com/help/deeplearning/ug/understand-network-predictions-using-occlusion.html>

Padding?? Important / **Bad** Info on Boarder

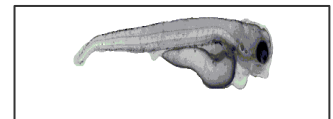
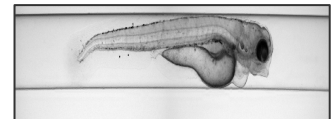
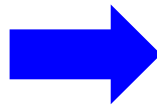
- Animal features frequently at edge of picture



AI 764, Fall 2020

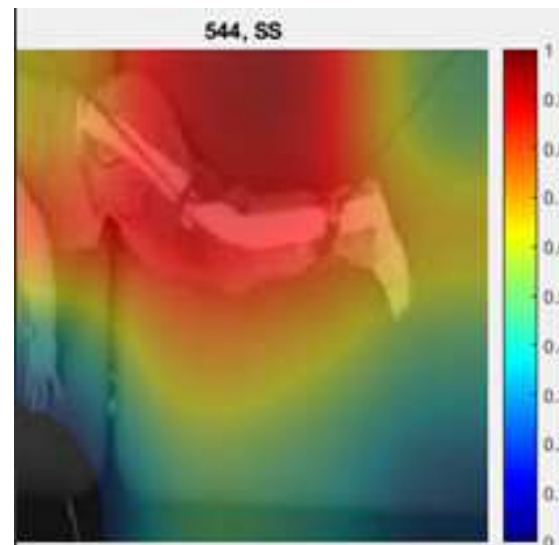
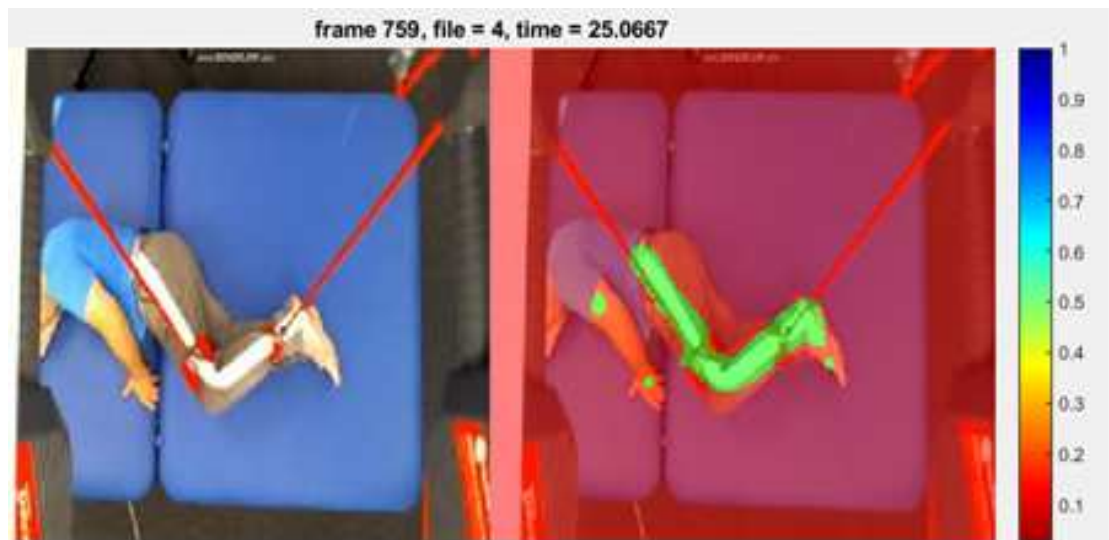
Kyle Widman,
Jake Park,
Pratiksh Patel,
Jacob Moen

- Removed footer— prevented training on timestamp
- How do we know if CNN learns something **BAD**?? **Check OS-Map!!**

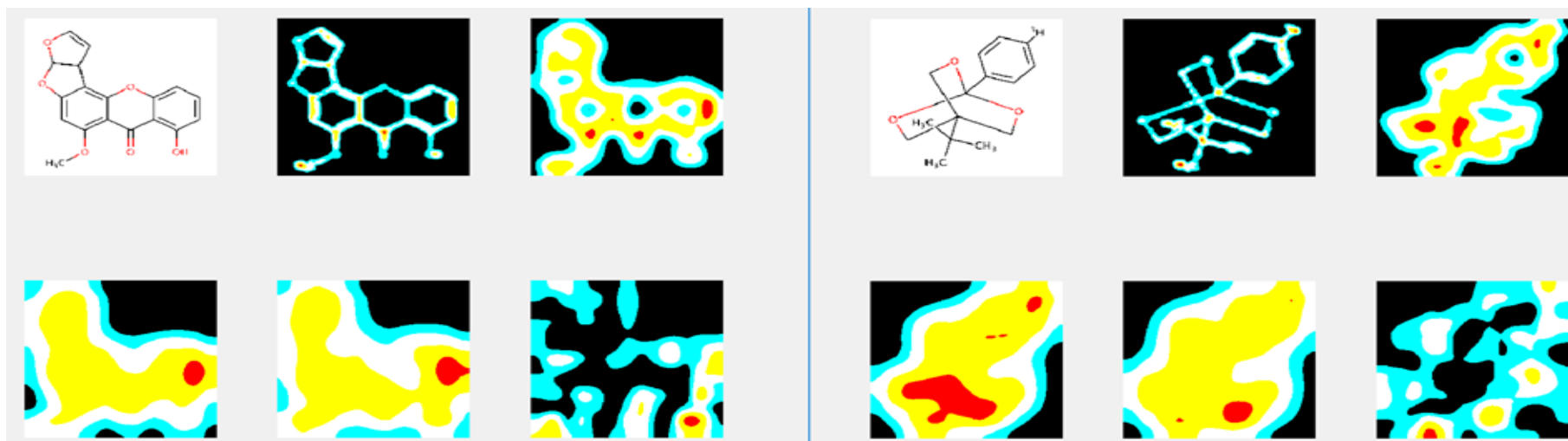


Surprising Facts Revealed by OS-Map

- Classifying motions into “strong”, “fair”, “weak”

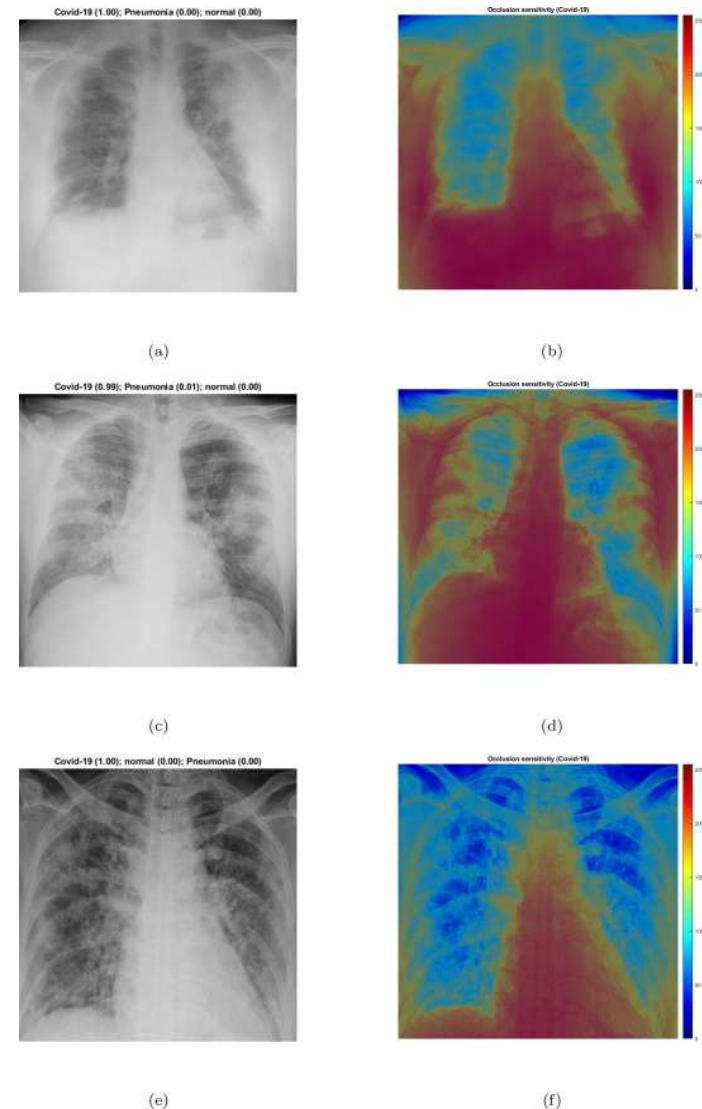


- Important Parts of Chemical Structure w.r.t Diseases



Covid-19 detection via deep neural network and *occlusion sensitivity maps*

- Covid-19 detection via deep neural network and *occlusion sensitivity maps* (PDF)
 - 2021..... in **Matlab** <https://www.sciencedirect.com/science/article/pii/S1110016821002076>



Lungs of Patients with COVID-19 vs Normal

Covid-19 CT Scan

764 AI, Fall 2020

John Kimutai

James Okullo

Zumari Chatham

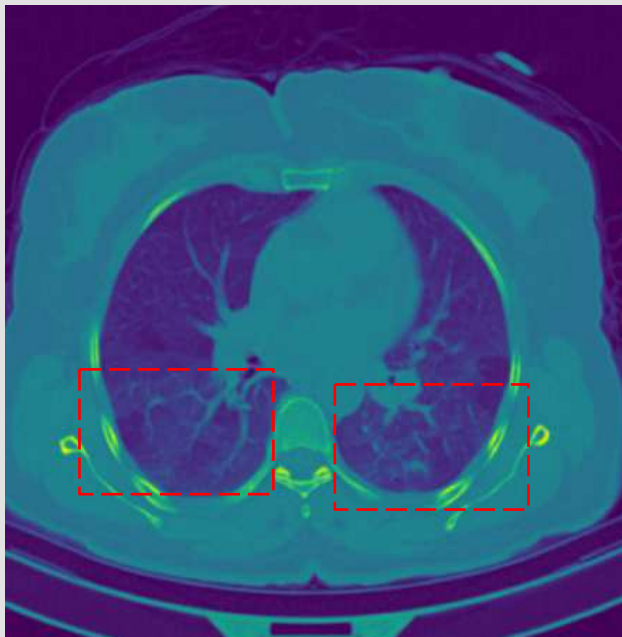
Niket Pandya

Youa Vang

- Can you tell the differences?

Lungs of patients with COVID-19 developed Ground Glass Opacities (GGO) lesions

COVID-19 Positive



Normal



TF-Explain & Grad_CAM

Covid-19 CT Scan

764 AI, Fall 2020

John Kimutai

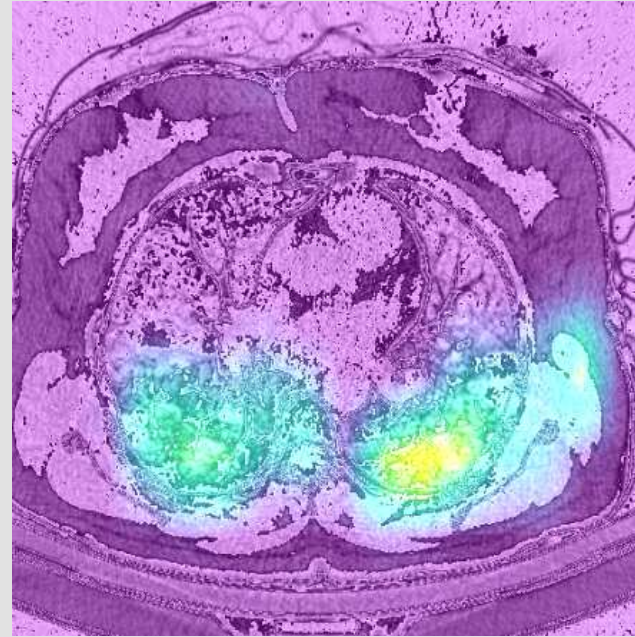
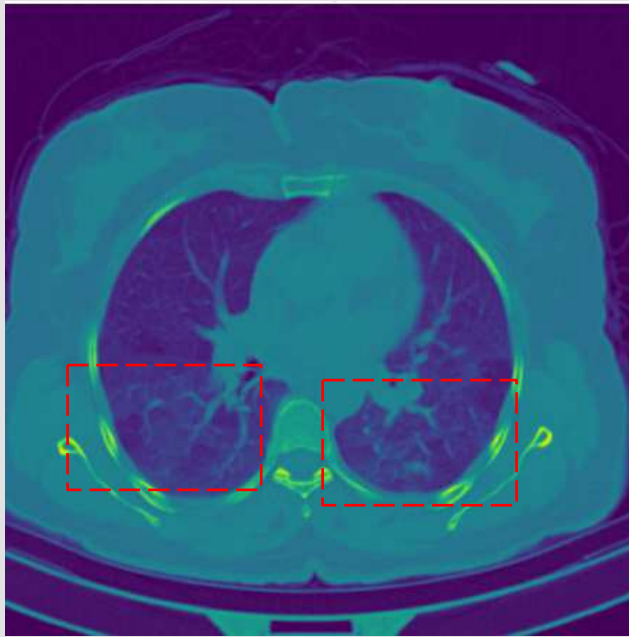
James Okullo

Zumari Chatham

Niket Pandya

Youa Vang

The Models Able To Detect Lesions In
Lungs of COVID-19 Positive Patients



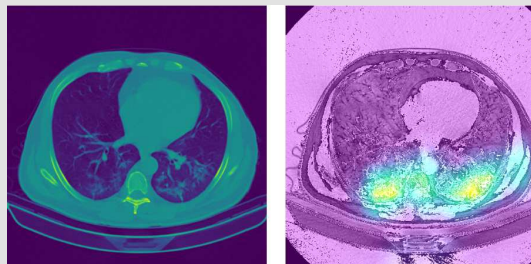
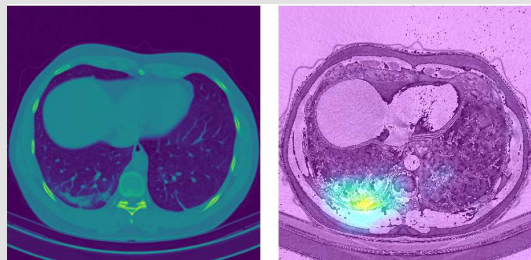
Covid-19 CT Scan

764 AI, Fall 2020

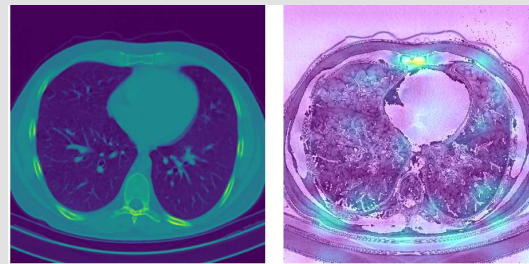
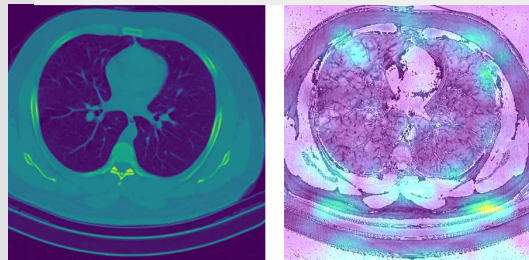
John Kimutai
James Okullo
Zumari Chatham
Niket Pandya
Youa Vang

Explain how the model identified TP, TN,
and FP images

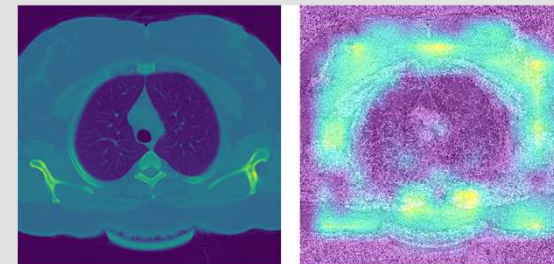
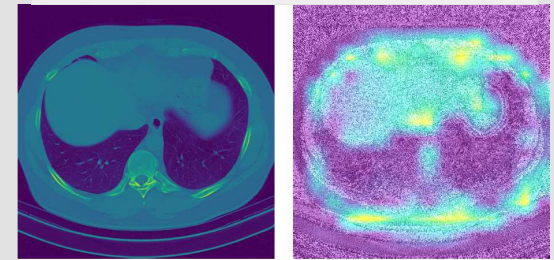
COVID-19 True Positive



Normal – True Negative



Normal – False Positive



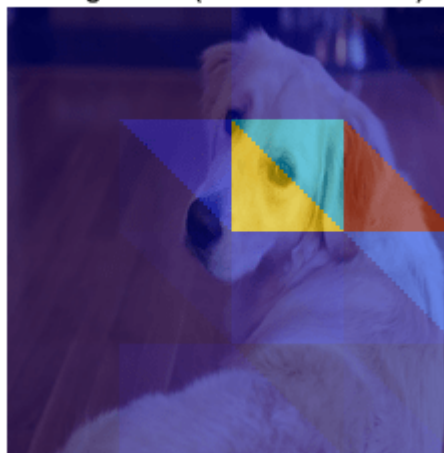
TF-Explain & Grad_CAM

Image LIME and Grad CAM

■ imageLIME

- <https://www.mathworks.com/help/deeplearning/ref/imagelime.html>
- <https://blogs.mathworks.com/deep-learning/2021/01/26/deep-learning-visualizations/>

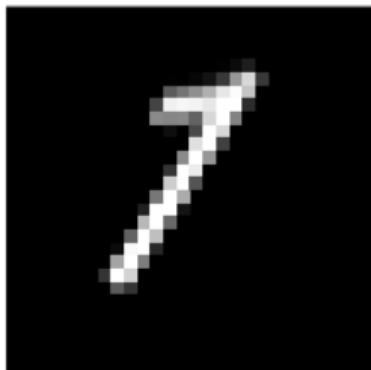
Image LIME (Golden Retriever)



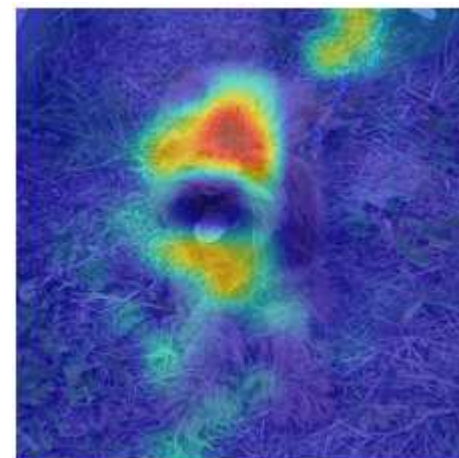
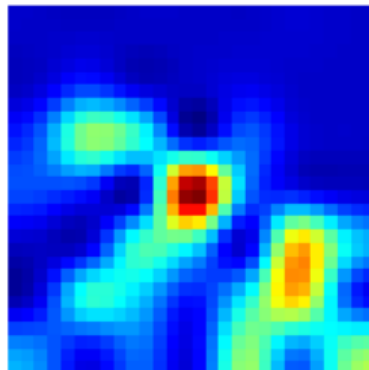
■ gradCAM

- <https://www.mathworks.com/help/deeplearning/ref/gradcam.html>
- <https://blogs.mathworks.com/deep-learning/2021/01/26/deep-learning-visualizations/>

True Rotation = 29
Pred Rotation = 27



Grad-CAM

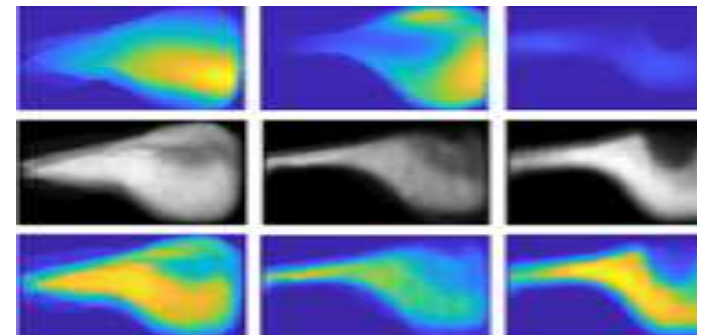
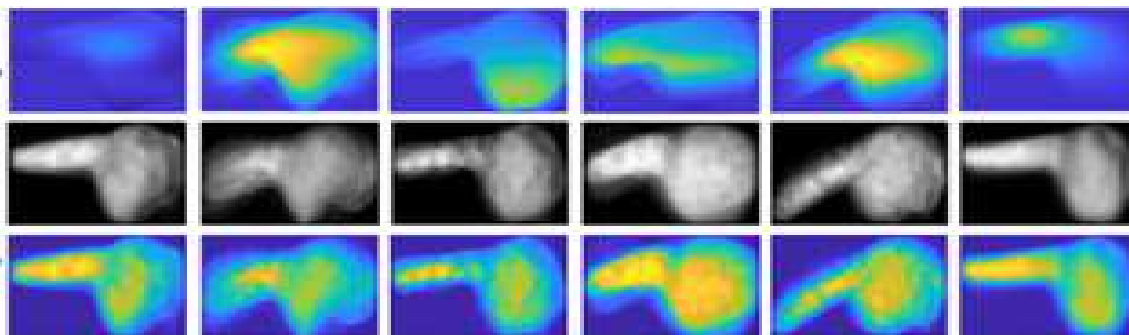
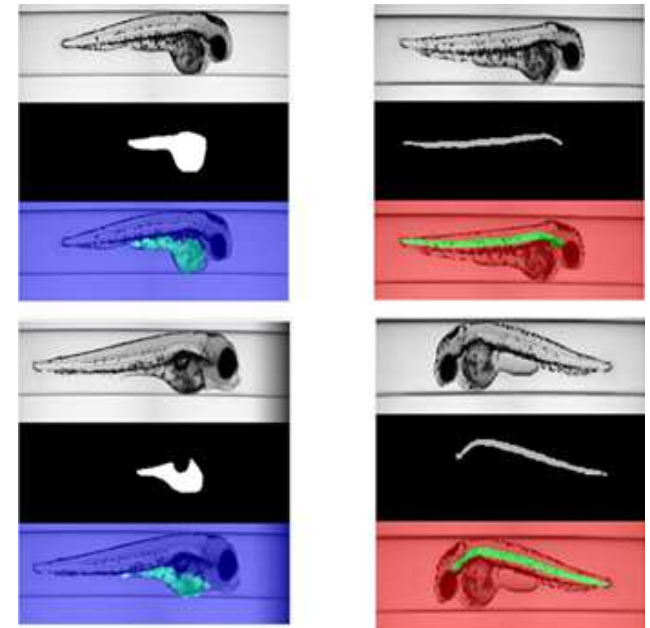


Case Study, Begin From Where??

- Find morphological changes in organs of fish embryos exposed to different toxins
- What are the **first TWO steps** in this analysis??

1. ???

2. ???

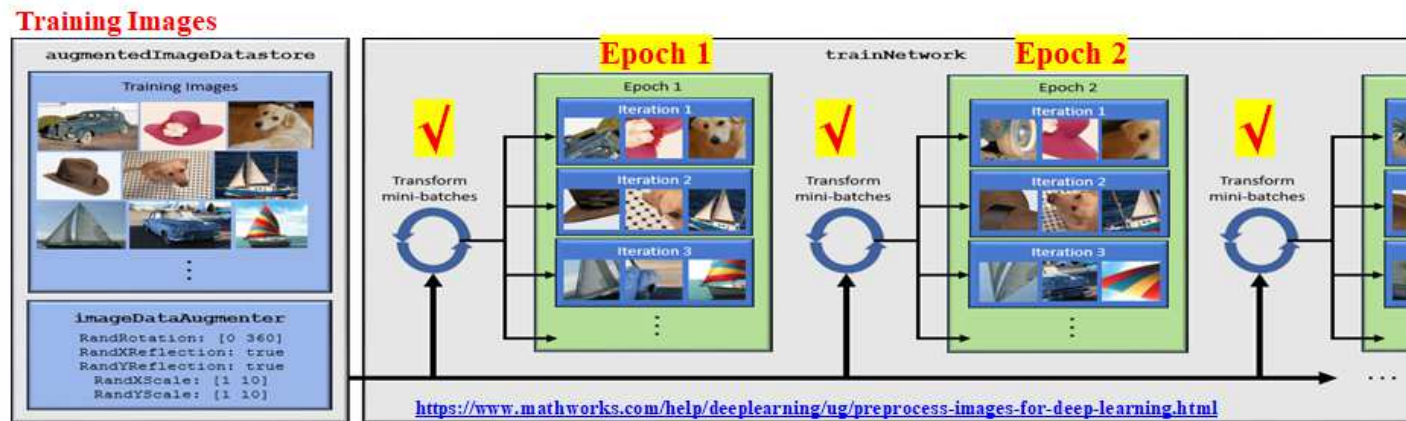


Outline

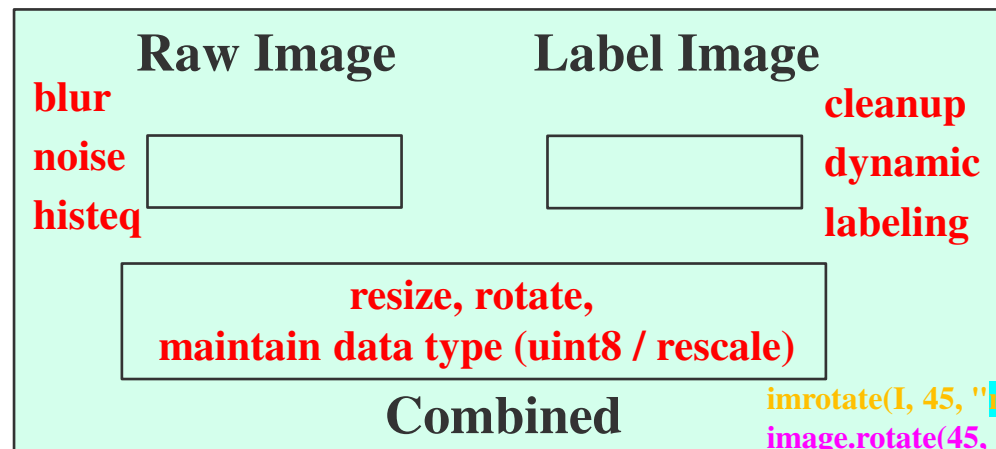
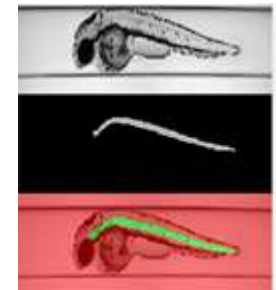
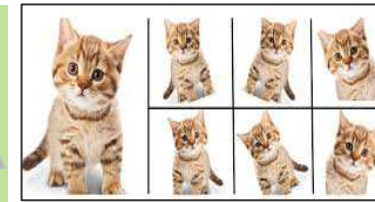
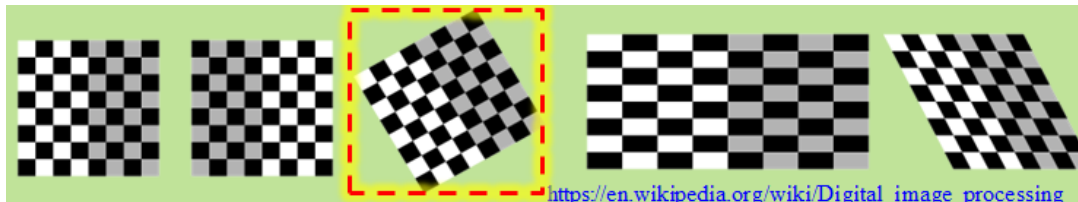
- Object Detection, Fast RCNN
- Object Detection, Semantic Segmentation, Labeling, Applications
- Encode + Decode CNN, Transposed Convolution, Pre-Trained CNN
- Quality Evaluation of Object Detection
-  **Augmentation Pitfalls**
- Identifying Morphological Changes in Organs w/ Occlusion Map

Anything Different? Image, Pixel, Split, Shuffle, Augment (Raw + Label)

HOW???



Simple Operations!!!

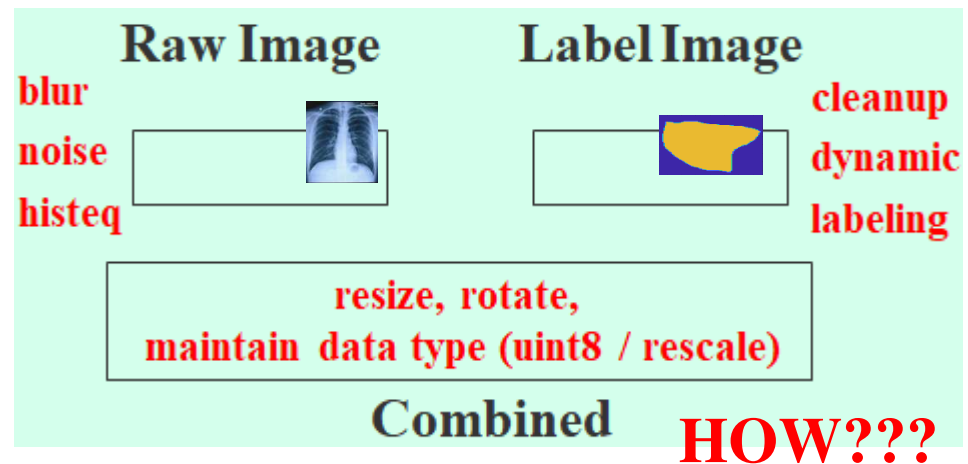


`imrotate(I, 45, "nearest")`
`image.rotate(45, resample=Image.NEAREST)`

Beyond CNN Architecture, Keras

Image, Pixel, Split, Shuffle, Augment (*Raw + Label*), Together

- Later **Augmentation Pitfalls!!!**



VAI_Slide_ImgPair_Prepare.ipynb

```
# 2. Create Dataset, sort so pairs are in SAME order, glob = wildcard name
image_paths = sorted(tf.io.gfile.glob(IMG_DIR + "*.png"))
mask_paths = sorted(tf.io.gfile.glob(MASK_DIR + "*.png"))
IR, IT, MR, MT= train_test_split(image_path, mask_path, 0.2)
# create "store" & tell TF to slice (image, mask) pairs together
DS = tf.data.Dataset.from_tensor_slices((image_paths, mask_paths))
DS = DS.map(load_and_preprocess) # process pairs using this func.
DS = DS.map(AUG) # AUG pairs using this func.
model.fit(train_DS, epochs=20, val_DS)
```

← create for Raw

← create for Label

← Split train / test of both

← Combine Raw + Label

← Read Pairs

← Augment both?

← Training

Prepare Image Pairs



Vision AI > Files > Course Examples + Data > VAI_Slide_FishEmbryo

FishEmbryo_All_Images.zip

FishEmbryo_Label_Images_BMP
FishEmbryo_Label_Images_PNG_for_Colab
FishEmbryo_Raw_Images_Yolk

VAI_Slide_ImgPair_Prepare.ipynb

```
def load_and_process(img_path, mask_path): # 1. read 1 img & 1 mask
    img = tf.io.read_file(img_path) # Read and decode raw images
    img = tf.image.decode_png(img, channels=3) # keras, do it explicitly
    img = tf.image.resize(img, IMG_SIZE)
    img = tf.cast(img, tf.float32) / 255.0 # Normalize to [0,1]

    mask = tf.io.read_file(mask_path) # Read and decode masks
    mask = tf.image.decode_png(mask, channels=1) # Labels are grayscale
    mask = tf.image.resize(mask, IMG_SIZE, method="nearest")
    return img, mask
```

```
# 2. Create Dataset, sort so pairs are in SAME order after glob
# glob = find wildcard names
image_paths = sorted(tf.io.gfile.glob(IMG_DIR + "*.png"))
mask_paths = sorted(tf.io.gfile.glob(MASK_DIR + "*.png"))
IR, IT, MR, MT = train_test_split(image_paths, mask_paths, 0.2)
# create "store" & tell TF to slice (image, mask) pairs together
DS = tf.data.Dataset.from_tensor_slices((image_paths, mask_paths))
DS = DS.map(load_and_preprocess) # process pairs using this func.
DS = DS.map(AUG) # AUG pairs using this func.
```

```
# 3. Final Pipeline to create train / val / test datastore
train_DS = (
    DS.shuffle(100) # sequential get from DS to fill buffer (100) for shuffle
    .batch(BATCH_SIZE)
    .prefetch(buffer_size=tf.data.AUTOTUNE) # get next BAT when current on GPU
)
```

```
def AUG(img, mask):
    if tf.random.uniform(()) > 0.5:
        img = tf.image.flip_left_right(img)
        mask = tf.image.flip_left_right(mask)
    img = bright(img, max_delta=0.1)
    return img, mask
```

```
model.compile( ..... 4.
model.fit(
    train_DS,
    epochs=20,
    val_DS # Prepared same way
)
```

TensorFlow 2.0
2020

Augment **BOTH** Images & Labels w/ Custom Function

```
from keras.preprocessing.image import ImageDataGenerator

def custom_augmentation_X(image): # or Y
    # Implement your custom augmentation logic w/ OpenCV or PIL

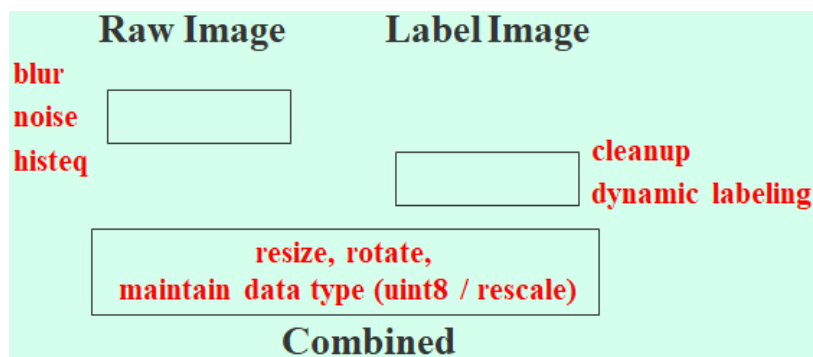
# 1. Create ImageDataGenerator w/ AUG for images
image_datagen = ImageDataGenerator(
    preprocessing_function = custom_augmentation_X, parameters)

# 2. Create ImageDataGenerator w/ AUG for labels
label_datagen = ImageDataGenerator(
    preprocessing_function = custom_augmentation_Y, parameters)

# 3. Use the flow method to apply AUG to BOTH images and labels
seed = 42 # Use same seed for both generators to ensure same AUG
image_generator = image_datagen.flow(images, seed=seed)
label_generator = label_datagen.flow(labels, seed=seed)

# 4. Combine generators using zip to
# ensure sync of augmented images and labels in batches
combined_generator = zip(image_generator, label_generator)

model.fit(combined_generator, ...)
```



Before TensorFlow 2.0

1. & 2. Create 2 image stores w/ unique aug func
 - **MUST duplicate common** ops in both funcs
3. Create 1 flow for sync purpose
4. Zip into 1 combined store for CNN

```
from sklearn.model_selection import train_test_split

# Split images and labels into training and testing sets
images_train, images_test, labels_train, labels_test =
    train_test_split(images, labels, test_size=0.2, random_state=42 )
```

Augmentation Pitfalls???

VAI_Slide_WHY_Care_Cases\Why_Care_Color_Look_Nice_or_Not.m

- Progressive annotation or bootstrapping segmentation
 - What's that?? After detecting easier objects...
 - Insert markers close to the object in input images
 - Guide training next SemSeg CNN on harder objects

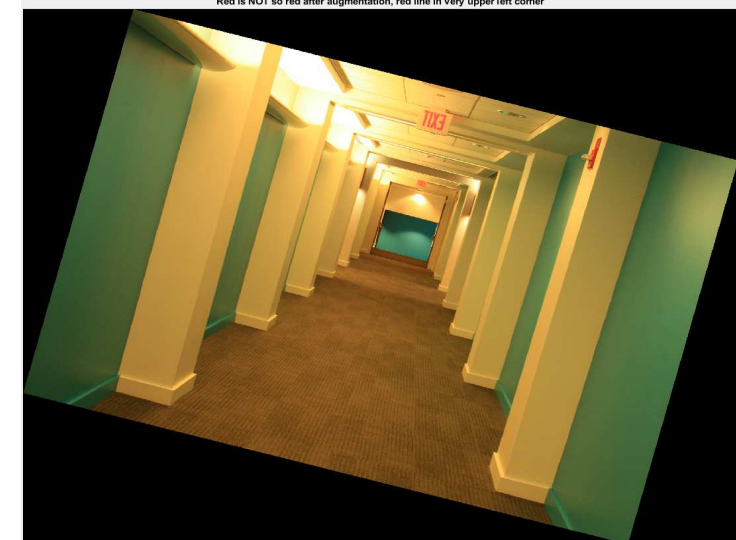
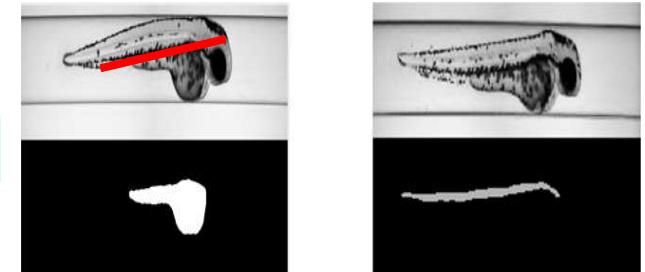


Figure 1, insert a red line w/ **smoothing**, but result is **not so red**

Figure 2, insert a red line **w/o smoothing**, result is **red**

Figure 3, after **AUG**, **not so red** (rotate, shear, xform)

Figure 4, after **AUG**, **still red** by store red locs before AUG

Figure 5, create label image with **red**

Figure 6, **resize 0.2 w/o “nearest”**, **not so red**

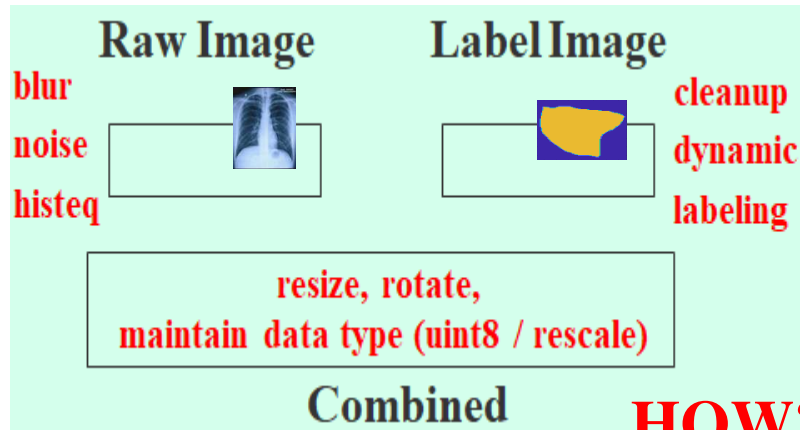
Figure 7, **resize 0.2 w/ “nearest”**, red **completely disappear**

For Figure 4 and 5

1. read in **input** image, add red lines, get “red pixel locs”
2. create a **temp** image with all black pixels
3. add a red line in **temp** image at “red locs”
4. apply desired transformation to **temp** image
5. find “**probably red**” pixel locations in **temp** img
6. make all “**probably red**” pixel to red in **input** image

Beyond CNN Architecture, **Matlab**

Image, Pixel, Split, Shuffle, Augment (*Raw + Label*), **Together**



HOW???

- Later **Augmentation Pitfalls???**

VAI_Slide_FishEmbryo\FishEmbryo_Semseg_Programs\

FishEmboyo_DeepLab_V3_Train.m

AUG_Img_AND_Label.m

FishEmboyo_DeepLab_V3_Test.m

Keras, 2 slides later

```
imds = imageDatastore(ImgFolder);           %% raw image datastore    ← create for Raw
pxds = pixelLabelDatastore(LabelFolder, ClassNames); %% label datastore ← create for Label

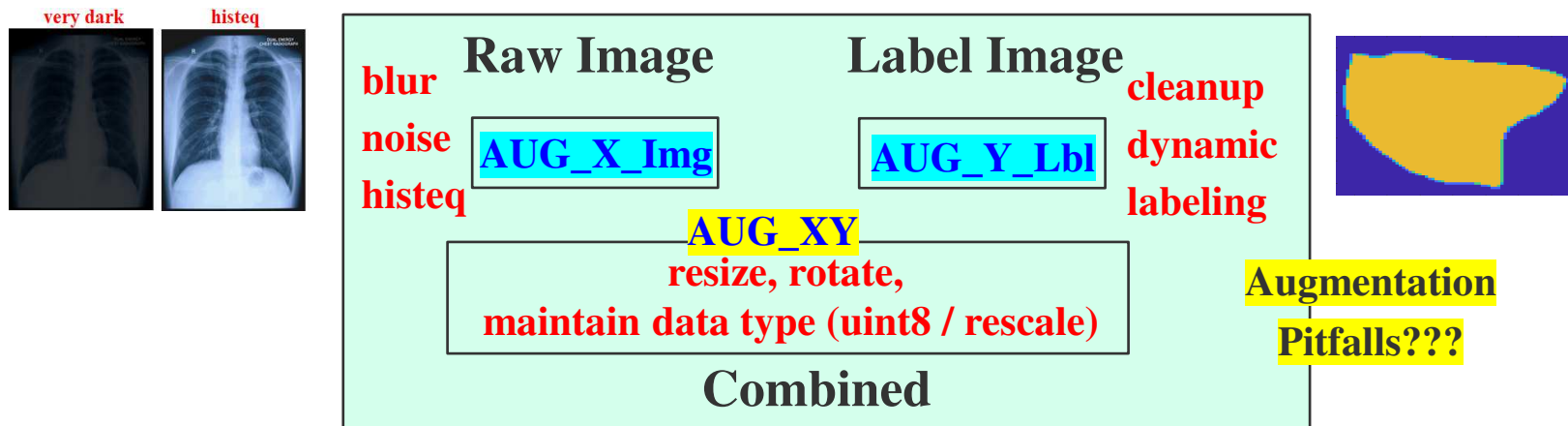
%% split training / testing on both images & labels
[imdsTrain, imdsVal, imdsTest, pxdsTrain, pxdsVal, pxdsTest, testIdx] = ...
    partitionCamVidData(imds, pxds, [0.8 0.0 0.2], obj);    ← Split train / test of both

dsTrain = combine(imdsTrain, pxdsTrain); %% combine together for sync ops ← Combine Raw + Label
dsTrain = transform(dsTrain, @(data) AUG_and_Img_Label(data)); %% aug both ← Augment both, example?
[CNN, info] = trainNetwork(dsTrain, lgraph, options);      ← Training
```

<https://www.mathworks.com/help/deeplearning/ug/semantic-segmentation-using-deep-learning.html>

Image + Pixel + Split + Augmentation (Raw + Label), **Separated**

- Create **2** image stores (generators), **BUT**... put **unique** ops in **EACH** augmentation program
- Create **1** combine store and add **common** operations in **ONE** augmentation program



```

Img_DS = imageDatastore(ImgFolder_1); % 1. add unique AUG ops
Img_DS.ReadFcn = @(filename)AUG_X_Img(filename, parameters);

Lbl_DS = pixelLabelDatastore(LabelFolder); % 2. add unique AUG ops
Lbl_DS.ReadFcn = @(filename)AUG_Y_Lbl(filename, parameters);

-----
% 3. combein X & Y into 1 store
Comb_DS = combine(Img_DS, Lbl_DS);

% 4. add common AUG ops
Comb_DS = transform(Comb_DS, @(data)AUG_XY(data, parameters));
    
```

```

% Augment common ops to raw & label images
function data = AUG_XY(data)
RawImg = data{1};
LblImg = data{2};

% do whatever AUG to raw & label images
% from 224×224×1 to 224×224×3
RawImg = cat(3, RawImg, RawImg, RawImg);

% more augmentation one RawImg & LblImg
data{1} = imresize(RawImg, [224 224], 'nearest');
data{2} = imresize(LblImg, [224 224], 'nearest');
    
```


Appendix

- Dice score for evaluating object identification results

Dice Score $\approx$ F1

<https://www.mathworks.com/help/vision/ref/nnet.cnn.layer.dicepixelclassificationlayer.html>

- The **Dice coefficient** can be used to **compare the pixel-wise agreement between a predicted segmentation and its corresponding ground truth**. where X is the predicted set of pixels and Y is the ground truth. The Dice coefficient is defined to be 1 when both X and Y are empty. The formula is given by:

$$2 * |X \cap Y| / |X| + |Y|$$

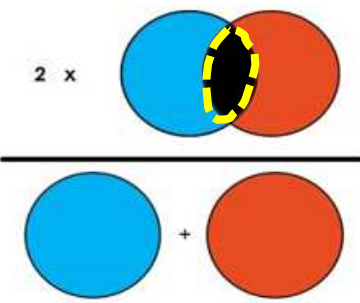
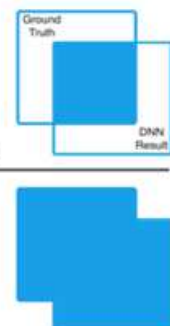
- **F1/Dice Score**: - Mean of the Dice coefficients for each <Image, Label> pair in the test set (3698*4 rows).

$$F1 / \text{Dice}: 2TP / 2TP + FP + FN$$

- **Why not IoU?**: In general, the **IoU metric tends to penalize single instances of bad classification more than the F score quantitatively even when they can both agree that this one instance is bad**. Similarly to how L2 can penalize the largest mistakes more than L1, the IoU metric tends to have a "squaring" effect on the errors relative to the F score. So the **F score** tends to measure something closer to **average** performance, while the **IoU** score measures something closer to the **worst**-case performance.

$$\text{IoU} / \text{Jaccard}: TP / TP + FP + FN$$

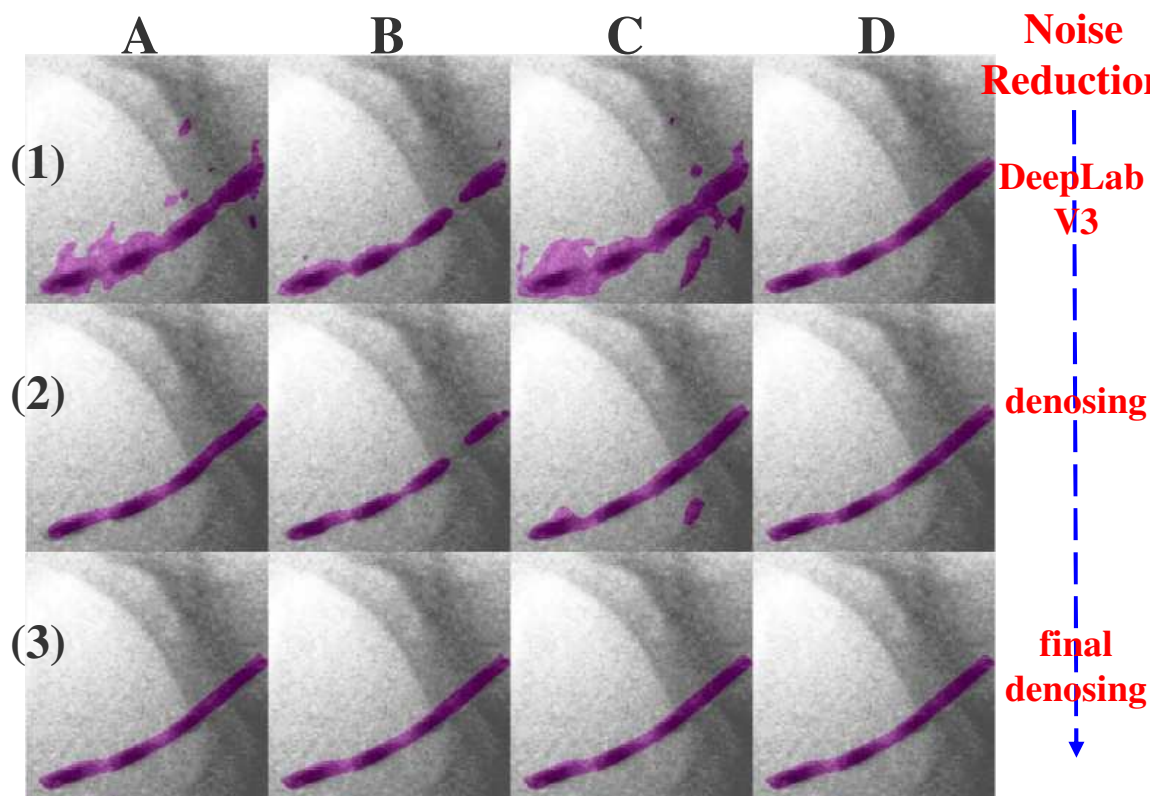
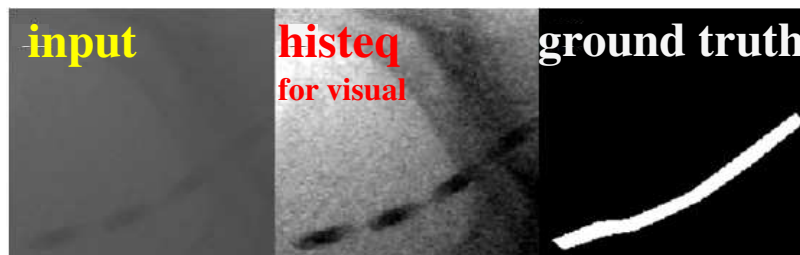
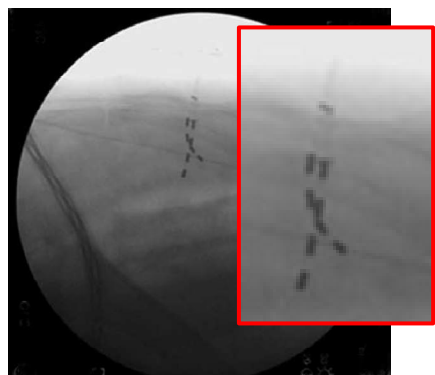
F.Y.I.

Confusion Matrix			Dice Score		IoU		
Predicted Values	Actual Values		$2 \times$		$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$		
	Positive (1)	Negative (0)					
	Positive (1)	TP					FP
	Negative (0)	FN					TN

2019 Fall,
Himanshu,
Alfi Goyal,
Rijesh Kumar,
Jerin Joseph

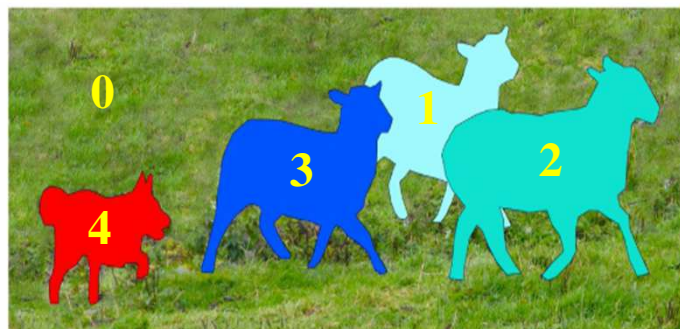
Noisy Reduction **WITHOUT** Traditional Image Processing

AI-Based Denoising and Identification of Neuromodulation Lead Locations from Limited Intraoperative Fluoroscopy Images

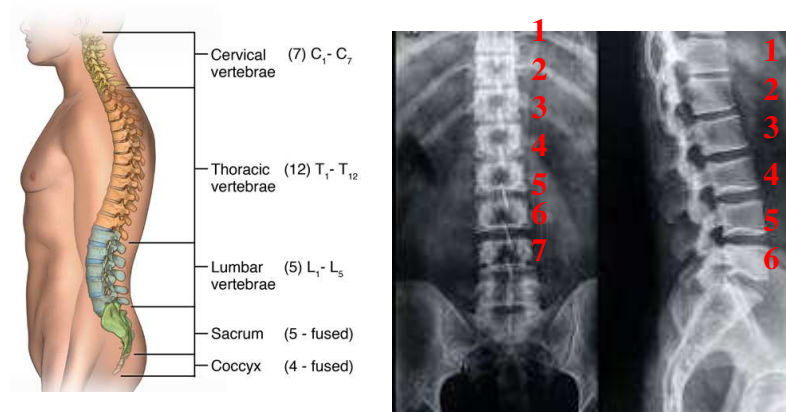


	A	B	C	D
IoU (1)	0.43	0.65	0.37	0.79
IoU (2)	0.80	0.74	0.69	0.88
IoU (3)	0.90	0.91	0.91	0.92
$\frac{(2) - (1)}{(1)}$	86%	14%	87%	11%
$\frac{(3) - (1)}{(1)}$	109%	40%	146%	17%

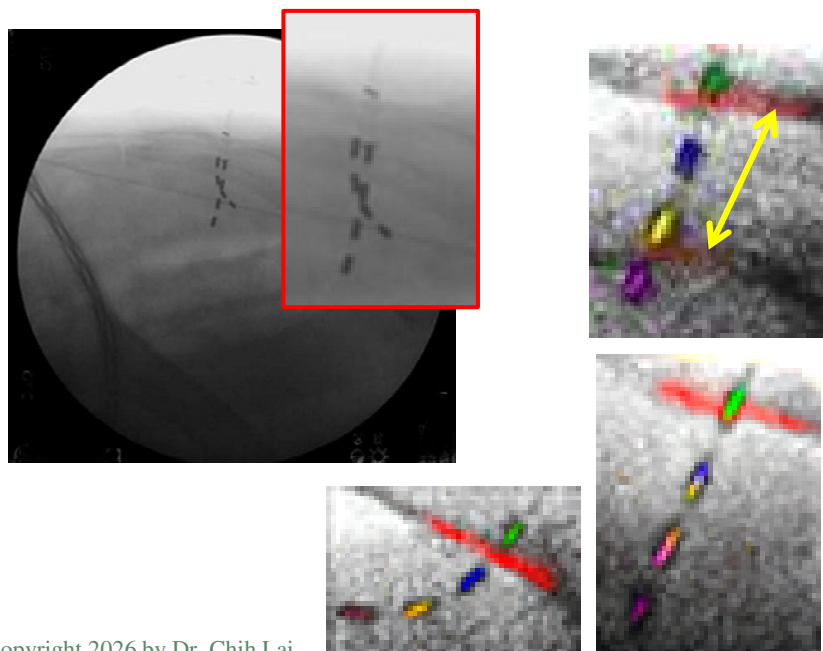
Instance Segmentation for Separate Identification



Instance Segmentation



- We end up using different methods to work around this problem ($\approx 0.4\text{mm}$)



Generative AI Approach

